

Тема 3 - Анализ на алгоритми.

Когато се пише алгоритъм за програма или подпрограма, програмистът трябва да има впредвид какви ресурси ще са нужни за изпълнението на програмата.

Двата основни ресурса, които трябва да се имат впредвид, са **времето и паметта**.

Това са отделни ресурси и в зависимост от ситуацията може в крайна сметка да изберете алгоритъм, който използва повече от един ресурс, за да спести от другия.

Анализът на ресурсите, които се използват от алгоритмите, позволява да се създава по-добър код!

Ресурсът, който избираме да оптимизираме с по-висок приоритет зависи от спецификите на задачата (приложението) и от изчислителната мощност на система.

Трябва ли програмата да завърши изпълнението си в рамките на ограничен период от време?

Системата има ли ограничено количество памет?

Може да няма един правилен избор. Важно е да се разберат плюсовете и минусите на всеки алгоритъм и избраната структура от данни за даденото приложение.

Тема 3 - Анализ на алгоритми.

Количеството консумирани ресурси често зависи от количеството данни, които ще бъдат обработвани.

Интуитивно е логично, че ако има повече данни за обработка, ще трябва повече място за съхранение на данните. Освен това ще отнеме повече време за изпълнение на алгоритъма.

Анализа на алгоритмите не отговаря на въпроса „Колко ресурс се изразходва за обработка на n порции от данни“... истинският въпрос, на който отговаря, е „Колко повече от същия ресурс ще изразходва за обработка на $n+1$ порции от данни “.

С други думи, това, което наистина ни интересува, е скоростта на нарастване на потреблението на ресурси по отношение на размера на данните.

Тема 3 - Анализ на алгоритми.

За функция се казва, че е монотонно нарастваща, ако нейната графика нараства само с увеличаване на стойностите на уравнението.

- * $O(n)$ означава, че функцията расте линейно, когато n расте.

$$f(n) = n+1$$

- * $O(n^2)$ означава, че функцията расте експоненциално, когато n расте

$$f(n) = n^2+2n +1$$

- $O(1)$ означава, че функцията не расте, когато n расте.

$$f(n) = 5$$

Тема 3 - Анализ на алгоритми.

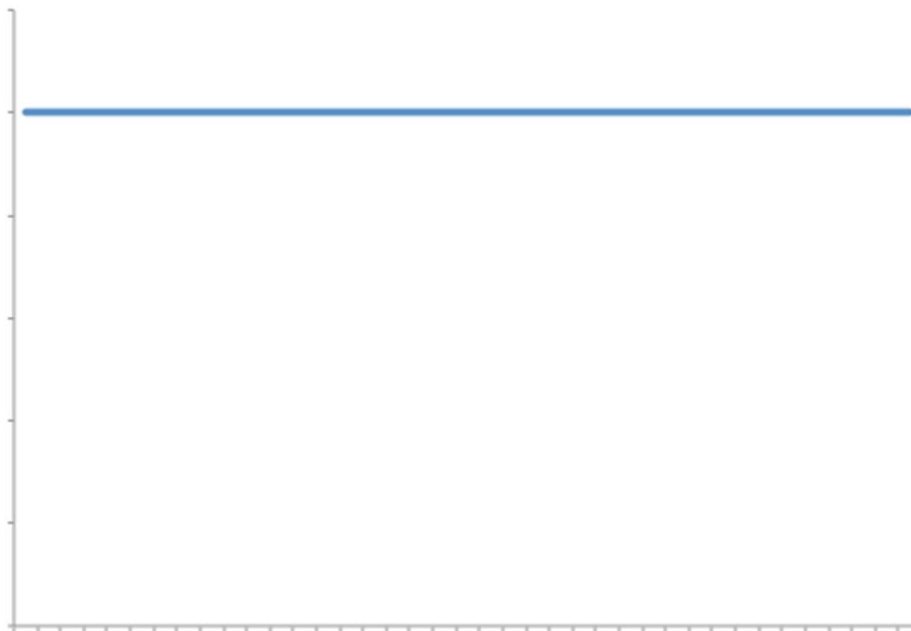
Темп на растеж (Growth Rates)

Анализът на алгоритмите е свързан с разбирането на темповете на растеж. Тоест, когато количеството данни става по-голямо, колко повече ресурс ще изисква изпълнението на алгоритъма? Обикновено ние описваме скоростта на нарастване на ресурсите на част от кода по отношение на функция.

Тема 3 - Анализ на алгоритми.

Постоянен темп на растеж (Constant Growth Rates) - $O(1)$

Постоянен темп на растеж е този, при който нуждата от ресурси не нараства. Това означава, че обработката на 1 порция от данните отнема същото количество ресурс като обработката на 1 милион порции от данни. Графиката на такъв темп на растеж изглежда като хоризонтална линия.



Тема 3 - Анализ на алгоритми.

Постоянен темп на растеж (Constant Growth Rates) - $O(1)$

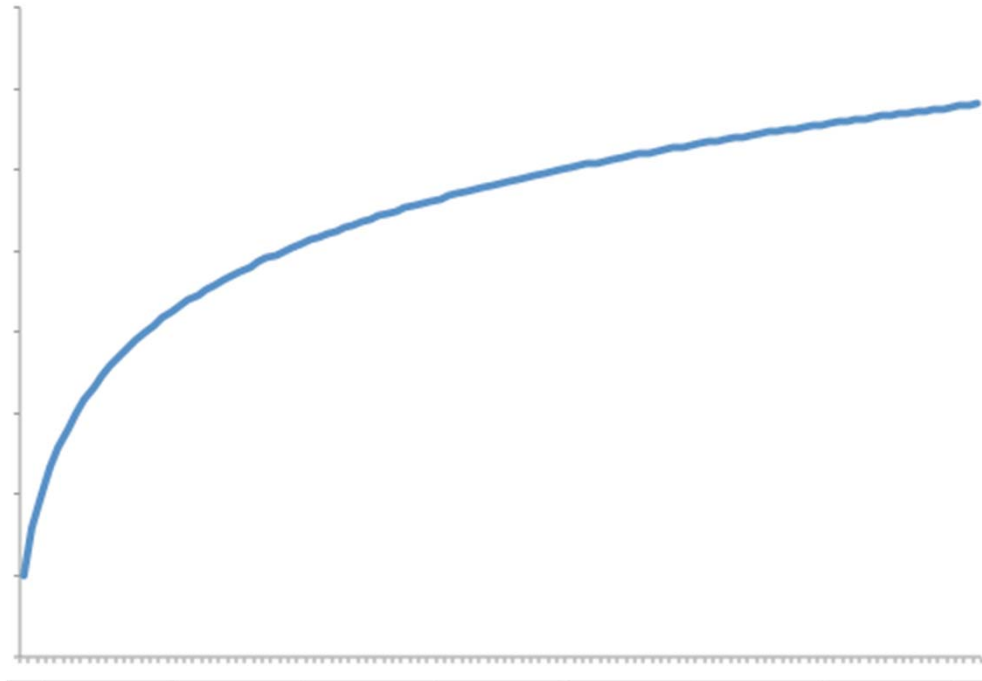
```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World";
    return 0;
}
```

Тема 3 - Анализ на алгоритми.

Логаритмичен темп на растеж (Logarithmic Growth Rates) - $O(\log n)$

Логаритмичен темп на растеж е темп на растеж, при който нуждите от ресурси нарастват с една единица всеки път, когато данните се удвоят. Това на практика означава, че с нарастването на количеството данни кривата, описваща скоростта на растеж, става по-плоска (по-близо до хоризонталната, но никога не я достига). Следващата графика показва как би изглеждала крива от този вид.



Тема 3 - Анализ на алгоритми.

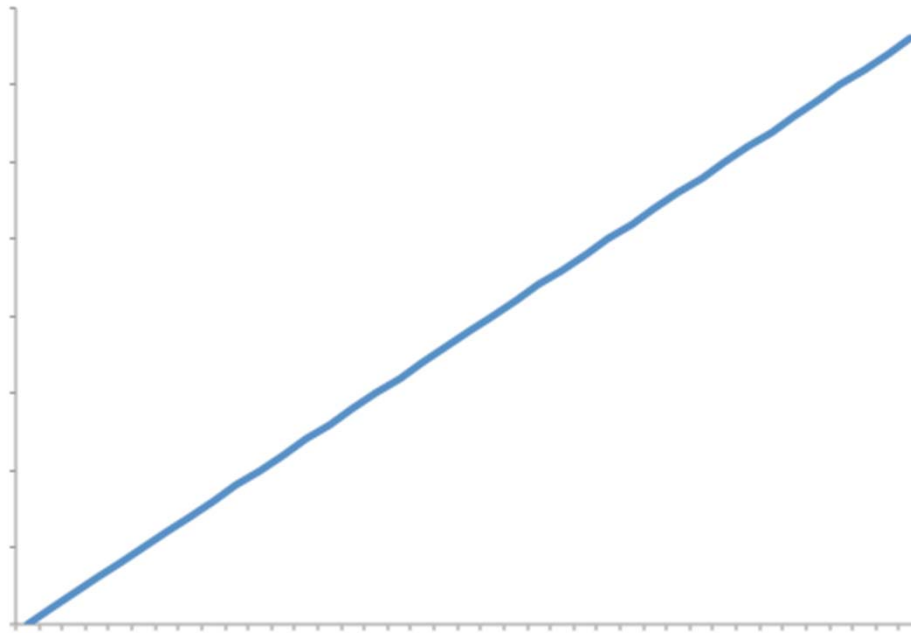
Логаритмичен темп на растеж (Logarithmic Growth Rates) - $O(\log n)$

```
beg = 0;
end = size - 1;
while ( beg <= end)
{
    // calculate mid value
    mid = (beg + end) / 2;
    /* if the specified element is found at mid index, terminate the process and return the index. */
    // Check middle element is equal to the defined element.
    If (arr[mid] == num)
    {
        return mid + 1;
    }
    else if (arr[mid] > num)
    {
        End = mid - 1;
    }
    Else if (arr [mid] < num)
    {
        Beg = mid + 1;
    }
}
// If the element does not exist in the array, return -1.
Return -1;
```


Тема 3 - Анализ на алгоритми.

Линеен темп на растеж (Linear Growth Rates) - $O(n)$

Линейният темп на растеж е темп на растеж, при който нуждите от ресурси и количеството данни са право пропорционални едно на друго. Тоест темпът на растеж може да се опише като права линия, която не е хоризонтална.



Тема 3 - Анализ на алгоритми.

Лиенеен темп на растеж (Linear Growth Rates) - $O(n)$

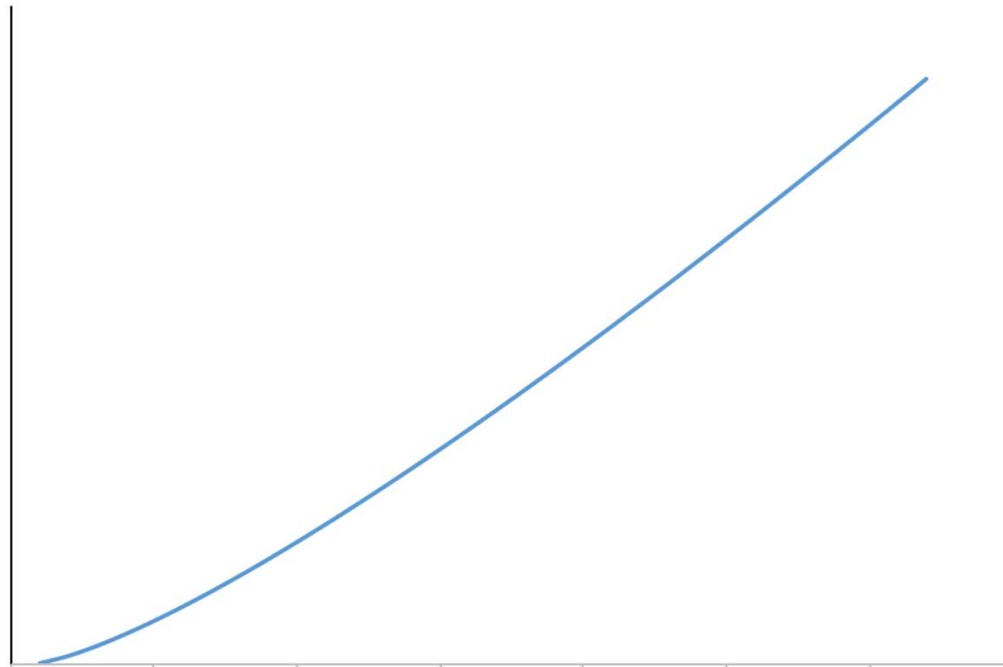
```
#include <iostream>
using namespace std;

int main()
{
    int i, n = 8;
    for (i = 1; i <= n; i++) {
        cout << "Hello World !!!\n";
    }
    return 0;
}
```

Тема 3 - Анализ на алгоритми.

Логаритмично-линееен темп на растеж (Log Linear Growth Rates) - $O(N * \log N)$

Логаритмично-линееен темп на растеж е леко извита линия. кривата е по-изразена при по-ниски стойности, отколкото при по-високи.



Тема 3 - Анализ на алгоритми.

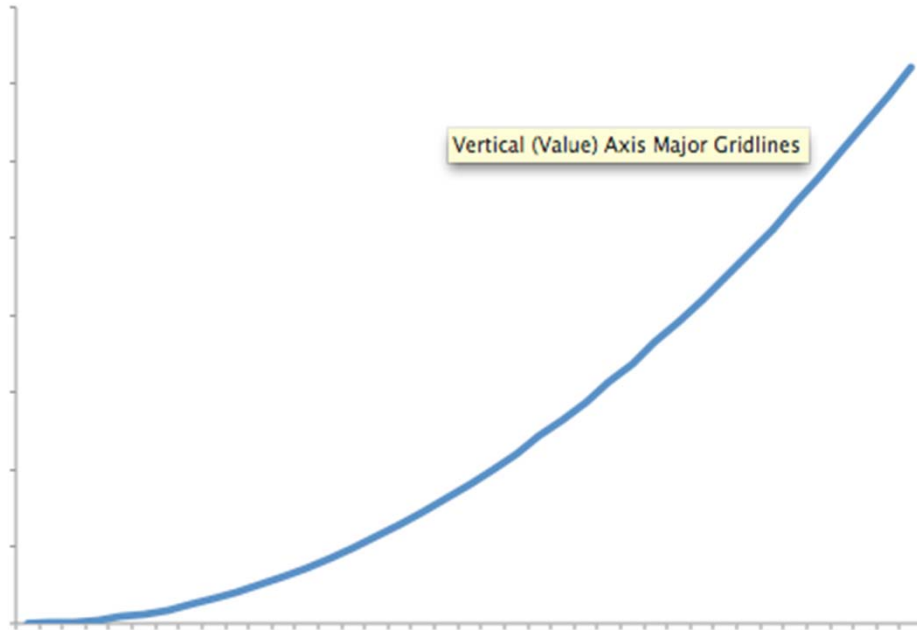
Логаритмично-линееен темп на растеж (Log Linear Growth Rates) - $O(N * \log N)$

Например: Да речем, че имаме дълъг сортиран списък с размер N . И имам Q числа, за всяко от тези Q числа трябва да намеря индекса му в дадения списък.

Тема 3 - Анализ на алгоритми.

Квадратичен темп на растеж (Quadratic Growth Rates) $O(N^2)$

Квадратната времева сложност представлява алгоритъм, чиято производителност е право пропорционална на размера на квадрата на набора от входни данни. В рамките на програмите тази времева сложност ще възникне всеки път, когато има вложени множество итерации в наборите от данни.



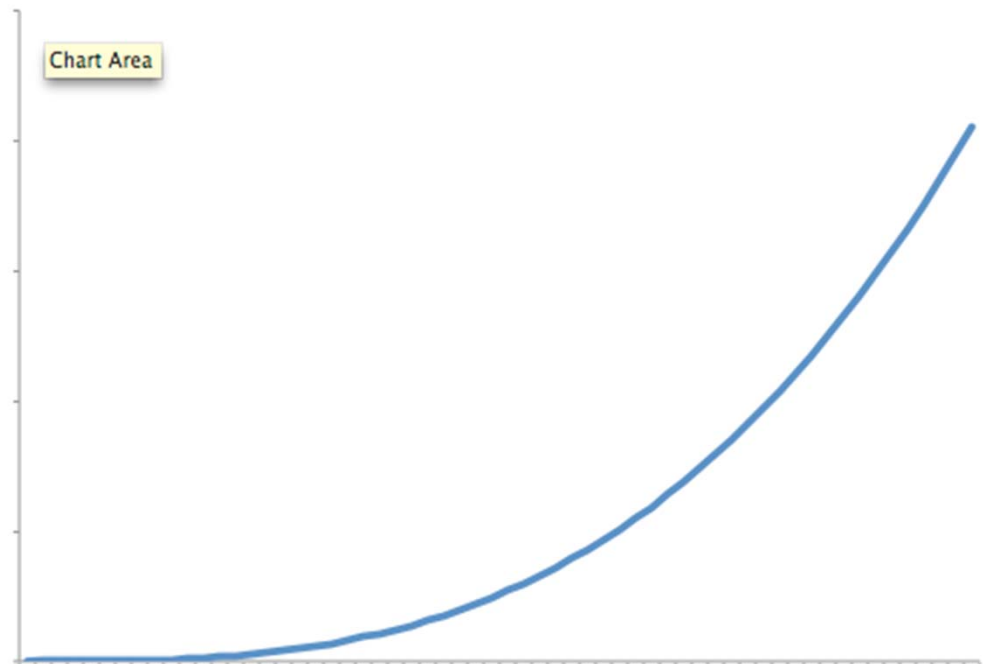
```
for i in N:  
  for j in N:  
    # Some processing
```

Тема 3 - Анализ на алгоритми.

Кубичен темп на растеж (Cubic Growth Rates) - $O(N^3)$

Казва се, че даден алгоритъм работи с кубичен темп на растеж, ако времето за изпълнение на трите цикъла е пропорционално на куба от N . Когато N се утрои, времето за изпълнение се увеличава с $N * N * N$.

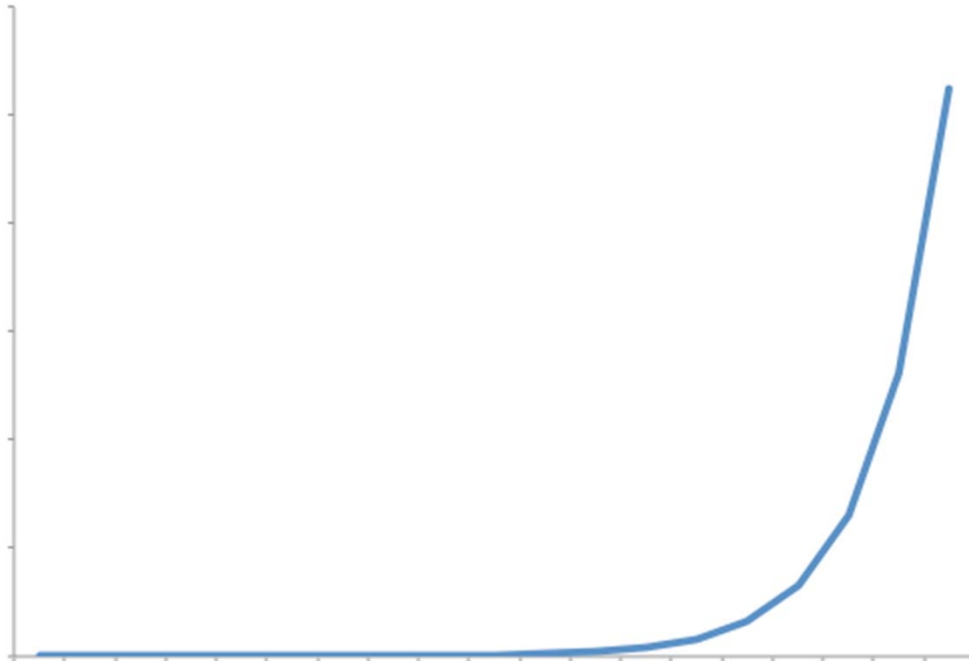
```
function findXYZ(n) {  
  const solutions = [];  
  
  for(let x = 0; x < n; x++) {  
    for(let y = 0; y < n; y++) {  
      for(let z = 0; z < n; z++) {  
        if( 3*x + 9*y + 8*z === 79 ) {  
          solutions.push({x, y, z});  
        }  
      }  
    }  
  }  
  return solutions;  
}
```



Тема 3 - Анализ на алгоритми.

Експоненциален темп на растеж (Exponential Growth Rates) - $O(2^N)$

Експоненциален темп на растеж е такъв, при който всяка допълнителна единица данни изисква удвояване на ресурс. Както се вижда, скоростта на растеж започва да изглежда като плоска, но бързо се изстрелва до почти вертикална (имайте предвид, че всъщност не може да бъде вертикална).



```
def F(n):  
    if n == 0:  
        return 0  
    else if n == 1:  
        return 1  
    else:  
        return F(n-1) + F(n-2)
```

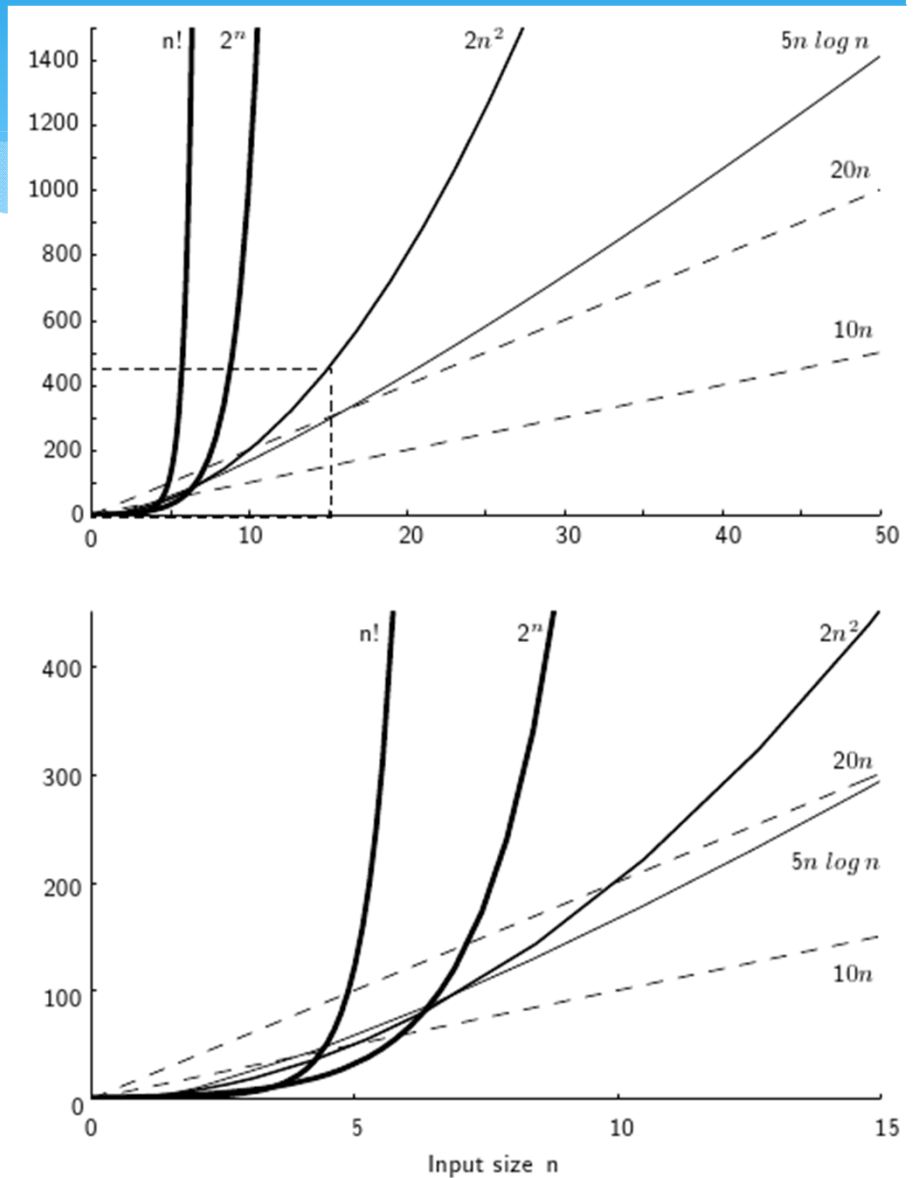
For every call to F, we make 2 more calls to F itself

Тема 3 - Анализ на алгоритми.

Факториал темп на растеж (Factorial Growth Rates) - $O(N!)$

Един от начините да мислим за факторните функции е, че те са спрямо експоненциалните функции това, което са квадратичните функции към линейните функции.

С други думи, те ги правят джуджета със скоростта, с която стават масивни. Това е в отделна категория, защото без значение колко голяма база избирате за вашия експоненциал, в крайна сметка факториелите ще наваксат изоставането, подобно на това как експоненциалите настигат квадратиците.



Тема 3 - Анализ на алгоритми.

$$1!=1$$

$$2!=2$$

$$3!=6$$

$$4!=24$$

$$5!=120$$

$$6!=720$$

$$7!=5040$$

$$8!=40320$$

$$9!=362880$$

$$10!=3628800$$

$$11!=39916800$$

$$12!=479001600$$

$$13!=6227020800$$

$$14!=87178291200$$

$$15!=1307674368000$$

$$16!=20922789888000$$

$$17!=355687428096000$$

$$18!=6402373705728000$$

$$19!=121645100408832000$$

$$20!=2432902008176640000$$

Тема 3 - Ефективност на алгоритми.

В компютърните науки алгоритмичната ефективност е свойство на алгоритъм, което се отнася до количеството изчислителни ресурси, използвани от алгоритъма.

Алгоритъмът трябва да бъде анализиран, за да се определи използваните от него ресурси, като ефективността на алгоритъма може да бъде измерена въз основа на това.

За максимална ефективност на алгоритъм е желателно да се сведе до минимум използването на ресурси. Различните ресурси, като времева и пространствена сложност обаче, не могат да се сравняват директно, така че кой от двата алгоритъма се счита за по-ефективен, често зависи от това коя мярка за ефективност се счита за най-важна в специфичната ситуация.

Тема 3 - Ефективност на алгоритми.

Нотация (система за означаване)	Асимптотична времева сложност	Пример
$O(1)$	константна	Намиране на медианата от сортиран списък с измервания; Използване на справочна таблица с постоянен размер; Използване на подходяща хеш функция за търсене на елемент.
$O(\log n)$	логаритмична	Намиране на елемент в сортиран масив с двоично търсене или балансирано дърво за търсене, както и всички операции в биномен куп.
$O(n)$	линейна	Намиране на елемент в несортиран списък или неправилно образувано дърво (в най-лошия случай) или в несортиран масив; Добавяне на две n -битови цели числа чрез пулсационно пренасяне.
$O(n \log n)$	линейна, логлинейна или квазилинейна	Извършване на бързо преобразуване на Фурие; heapsort, quicksort (най-добър и среден случай) или merge sort
$O(n^2)$	квадратна	Умножение на две n -цифрени числа по прост алгоритъм; bubble sort (в най-лошия случай или наивна реализация), Shell sort, quicksort (в най-лошия случай), selection sort or insertion sort
$O(c^n), c > 1$	експоненциална	Намиране на оптимално (неприближително) решение на задачата на пътуващия търговец чрез динамично програмиране; определяне дали две логически твърдения са еквивалентни с помощта на brute-force търсене

Тема 3 - Ефективност на алгоритми.

Бенчмаркинг (Benchmarking): измерване на ефективността

За нови версии на софтуер или за предоставяне на сравнения с конкурентни системи понякога се използват бенчмаркове, които помагат при измерване на относителната производителност на алгоритмите.

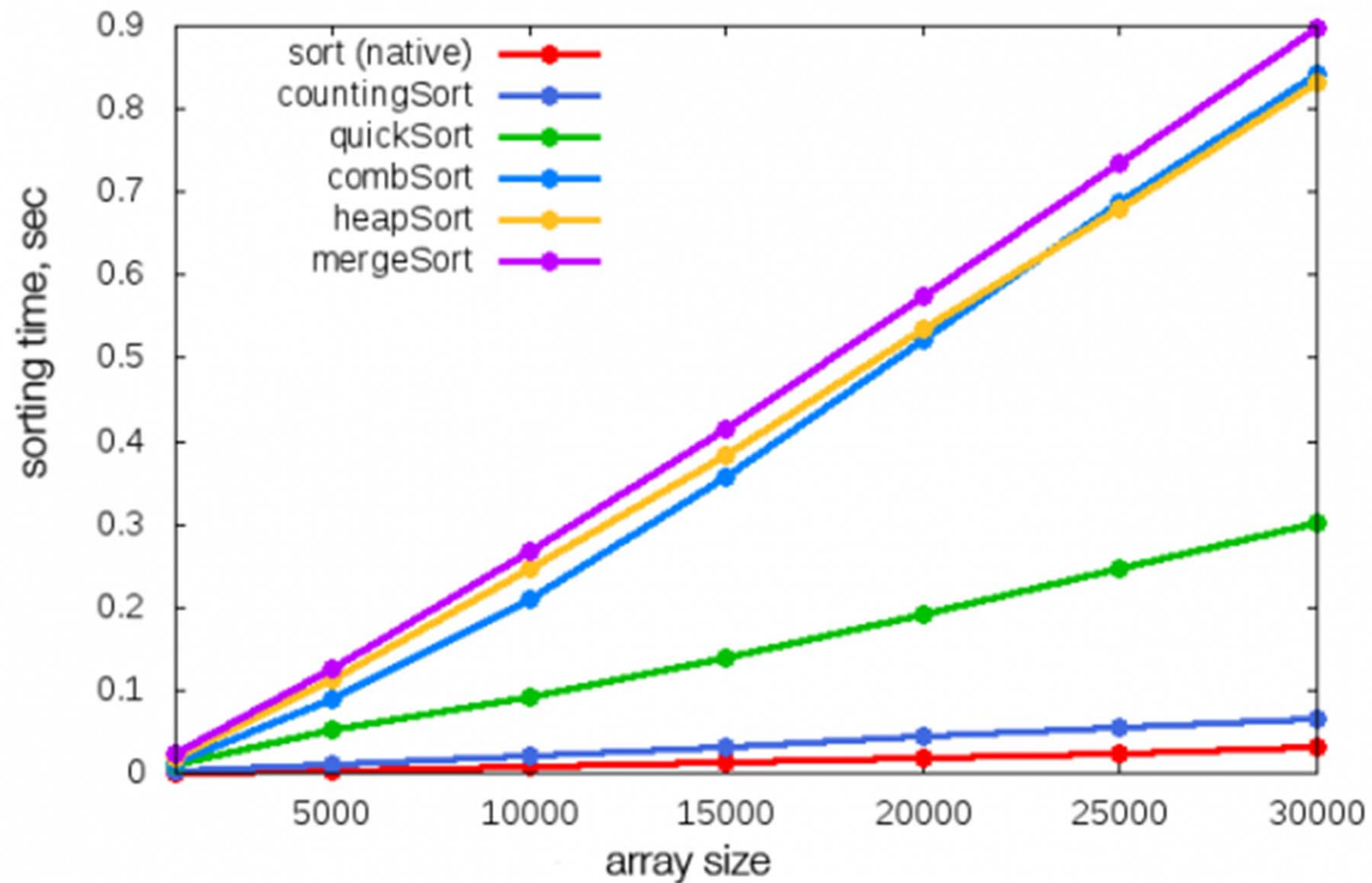
Ако се създаде нов алгоритъм за сортиране, например, той може да бъде сравнен със своите предшественици, за да се гарантира, че поне е ефективен както преди с известни данни, като се вземат предвид всички функционални подобрения.

Бенчмарковете могат да се използват от клиентите, когато сравняват различни продукти от алтернативни доставчици, за да преценят кой продукт ще отговаря най-добре на техните специфични изисквания по отношение на функционалност и производителност.

Някои бенчмаркове предоставят възможности за създаване на анализ, сравняващ относителната скорост на различни компилаторни и интерпретаторни езици.

Тема 3 - Ефективност на алгоритми.

Sorting array of INTs not exceeding the size of array
Size of array - up to 30 thousand elements



Тема 3 - Оптималност на алгоритми.

Следствие от това, че един алгоритъм е асимптотично оптимален, е, че за достатъчно големи входове никой алгоритъм не може да го надмине с повече от постоянен фактор.

Поради тази причина асимптотично оптималните алгоритми често се разглеждат като „края на линията“ в изследванията, постигането на резултат, който не може да бъде значително подобрен.

Обратно, ако даден алгоритъм не е асимптотично оптимален, това означава, че с нарастването на размера на входа алгоритъмът се представя все по-лошо от най-добрия възможен алгоритъм.

Тема 3 - Коректност на алгоритми.

Алгоритъмът е напълно коректен, ако получава валиден вход, прекратява работата си и винаги връща правилния изход.

Единственият начин да се докаже коректност на даден алгоритъм върху всички възможни входове е чрез формално или математическо разсъждение за него. Една форма на разсъждение е „доказателство чрез индукция“, техника, която също се използва от математиците за доказване на свойствата на числови последователности.

Тема 3 - Коректност на алгоритми.

За всеки алгоритъм трябва да докажем, че той винаги връща желан изход за всички правни случаи на проблема. За сортиране това означава дори ако входът вече е сортиран или съдържа повтарящи се елементи. Търсенето на контрапримери е най-добрият начин да се опровергае или докаже неговата коректност.